泰坦尼克号学习笔记

一、设置随机种子

- 设置的万能随机种子为42

二、读取数据

- 进行copy，以后处理副本，不破坏原数据。

三、数据预处理

1、特征选择(丢弃数据)

- PassengerId，单纯的编号，对于模型预测是否生存没有实际意义。
- Name，尽管其中的Mr, Mrs, Miss是有一些信息的，但是可以根据Sex和Age来判断，不必再提取Name中的信息了，而且Name是文本数据转化为数字较为困难。
- Ticket，对于预测结果没有实际意义，且格式难以统一。
- Cabin，数据缺失较高，即使填充为NAN，也会徒增偏差和噪音，对于训练没有什么帮助。

2、提取特征和标签

- 特征有Pclass、Sex、Age、Fare、Embarked
- 标签是Survived
- 特征矩阵 X 作为模型输入，标签向量 y 作为模型预测的目标结果。

3、划分数据集

- 进行二八分，并写上随机种子保证可复现性。

4、处理缺失值

- 定义preprocess_params用来存放计算出的所有关键参数。原因：题目要求测试数据和新样本必须复用同一套处理方式，所以既可以用来填充测试集的缺失，也方便后面的取用。
- 缺失值填充的参数(平均值，均差，众数等)只能根据训练集计算，算出参数后，再将它们应用于训练集和测试集，防止数据泄露。

5、构造新特征

- FamilySize=SibSp+Parch+1。模型并不能准确理解SibSp，Parch的含义，所以构造了一个新的特征家庭总人数来更加具体准确的说明同行的人数对于生存率的影响，并且删去SibSp和Parch这两个特征。
- 为什么在划分数据集之后：防止数据泄露。

6、类别特征转换

- 将字符串映射成数字编码。
- Sex：male映射成0，female映射成1。
- Embarked：C映射成0，Q映射成1，S映射成2.不使用独热编码是那样会增加特征维度，而本次数据集较小，使用简单的数字映射足以让模型学习到规律。

7、数值特征标准化

- 对Age和Fare两列进行标准化。
- 算法：(原数值-平均值)/标准差。
- 均值和标准差必须从训练集中计算，然后再应用到训练和测试集上。

8、保存预处理参数

- 将所有计算出的参数统一存入一个字典preprocess_params中，利用json库保存为data/preprocess_params.json文件。同时把最终的特征列feature_cols也存进去。

四、Dataset和DataLoader

1、Dataset

- 将Pandas DataFrame转换为PyTorch Tensor
- 对于y使用.view(-1,1)把形状从[N]一维张量变[N,1]二维张量，因为后续的BCELoss二分类交叉熵输出的形状是二维张量，所以要求标签形状和模型输出形状一致。

2、DataLoader

- batch_size=32，一次抓取32组数据。
- 先定义train_dataset，test_dataset来接受打包抓取的32组数据。
- 然后DataLoader，拿数据，打包，train打乱数据。test不打乱数据，保持可复现性。

五、构建自定义二分类模型

- 输入层nn.Linear(input_dim,16)，线性回归，六个特征扩展成16个维度的中间特征。
- 隐藏层nn.ReLU()，激活函数，把负数变为0，非线性变换。
- 输出层nn.Linear(16,1)，输出一个任意实数
- nn.Sigmoid()，输出激活函数，把输出的一个任意实数压缩到[0,1]之间，从而输出概率。

六、训练模型train_model

- 1、训练50轮，学习率设置为0.001(epochs=50,lr=0.001)
- 2、选用BCELoss二分类交叉熵损失函数，因为这是专门为二分类任务定制的损失函数。
- 3、优化器选用Adam，自动调整学习步长，返回模型中所需要的更新参数，而且对于这种比较小的数据集，Adam比SGD收敛更快，更稳定。
- 4、过程：梯度清零->前向传播->计算损失->反向传播->优化器调整参数。计算每一轮的损失和准确率。
- 5、切换为评估模式model.eval()
- 6、关闭梯度计算，进行测试，然后计算测试集准确率。
- 7、输出Train Loss，Train Acc,Test Acc。
- 8、画图，训练集的loss在一张上，训练集和测试集的准确率在一张上。

七、main.py和predict.py

1、main.py

- 数据处理process_data -> get_loader对数据进行封装 -> `input_dim = X_train.shape[1]`动态获取特征维度，将其输入到Model中 -? 开始训练。
- 在main.py中训练和预测是一起进行的，同时在train.py中`torch.save(model.state_dict(),'src/titanic_model.pth')`使在训练完后保存了训练后的权重w和偏置b，使得只需训练一次，就可以多次预测，不用每运行一次就要训练一次，提高效率以及具有可复现性。以及对应题目的“保存能够重新加载的模型，以及预测所需的预处理信息。”

2、predict.py

- 对于输入的样本进行和data_process同样的数据处理，填充缺失值的数据是data_process中根据训练集算出的数据。

八、实验配置和超参数

1、实验配置

- Windows+VS Code+Python(PyTorch虚拟环境)+cpu训练(数据集比较小)

2、超参数

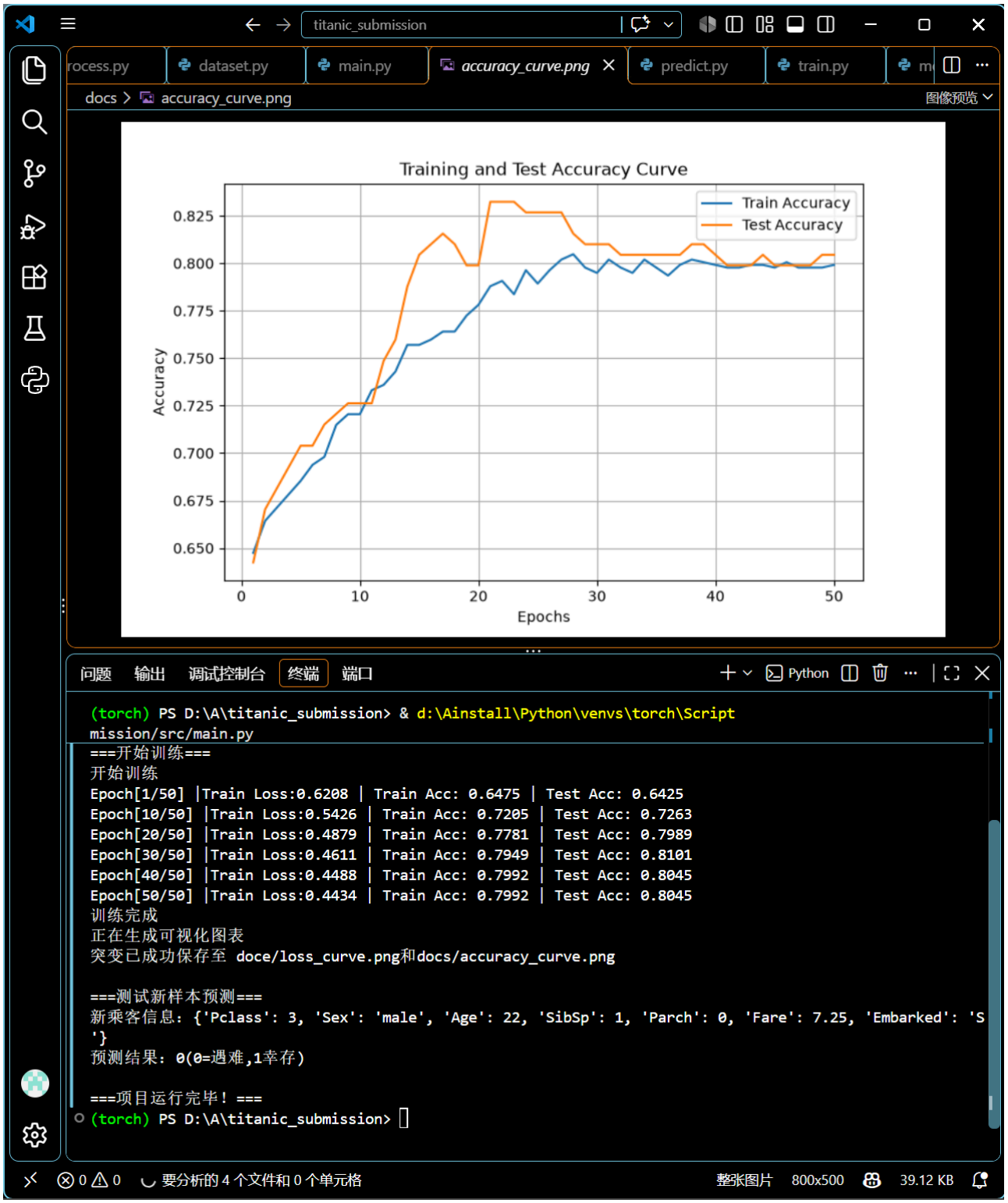
- test_size=0.2,二八和三七分训练出的准确率相差不大，但是二八分使得训练集的数据更多一点，模型可以更好的进行训练。
- random_state=42,万能种子。
- batch_size=32,train打乱顺序，test不打乱。
- 隐藏层神经元数量为16，到后面两条线基本贴合，说明拟合效果比较好，泛化能力比较强。为8时，Test_acc波动太大，而且会欠拟合。为32时，训练到二十轮时，训练集准确率跑到了测试集准确率上方，而且测试集准确率略有下降，发生了过拟合。
- epochs=50。
- lr=0.001,学习率为经典值，过大会震荡，过小会收敛慢。
- BCELoss()，二分类交叉熵损失函数
- Adam优化器

九、问题及解决方法

- (1)在处理缺失值时，执行`dataset['Age'].fillna(age_mean, inplace=True)`报错，改为`dataset['Age'] = dataset['Age'].fillna(age_mean)`。原因：最新的pamdas中使用`inplace=True`,不能对dataset进行修改，所以使用赋值的语法。
- (2)在定义损失函数BCELoss时写成`criterion=BCELoss`,但应该是`criterion=BCELoss()`
- (3)在写predict.py是漏写构造FamilySize新特征。

十、实验结果和心得体会

1、测试集准确率



2、结果分析

- Loss从0.6719降低到0.4390，Train Acc从0.6096提升到0.8048，Test Acc从0.6313提升到0.7877
- Loss曲线平滑下降并逐渐收敛。
- 两条准确率曲线相互交织，上升，后期趋于平稳，没有出现过拟合和欠拟合。

3、心得体会

- 数据预处理非常重要；有错不可怕，慢慢改；实现了从数据读取到新乘客预测的完整流程。